

Resumen de clase

Ejercicio Firewall

Ideas de Diseño y
Command Pattern



2° cuatrimestre 2008

Contenido

RESUMEN DE CLASE EJERCICIO FIREWALL.....	3
ENUNCIADO.....	3
EXPLICACIÓN DEL DOMINIO.....	4
PRIMERAS IDEAS.....	4
VALUE OBJECTS, GOD OBJECTS	6
MODELANDO LAS REGLAS.....	6
UNA SOLUCIÓN	7
COMBINANDO REGLAS	10
CONFIGURANDO LA REGLA PARA EL ENVÍO DE UN MENSAJE.....	11
MODELANDO EL ENVÍO DE UN PAQUETE (INTRO BREVE A INTERFACES)	12
LO BUENO Y LO MALO DE CONFIGURAR VARIAS ACCIONES	13
INTERFACES DEL SISTEMA	13
<i>Interfaz de entrada con el firewall.....</i>	<i>13</i>
<i>Interfaz de salida que usa el firewall.....</i>	<i>13</i>
EL COMMAND PATTERN.....	14

Resumen de clase Ejercicio Firewall

Enunciado

Se requiere modelar un firewall, que actúa en el punto de entrada a una LAN. Se controlarán sólo las comunicaciones entrantes, procedentes de una WAN.

El firewall recibe mensajes de la red WAN y los forwarda solamente a la LAN (o sea, que sólo maneja mensajes entrantes).

Debe permitir las siguientes funcionalidades:

Configuración y ejecución de reglas

Control de acceso a puertos: debe permitir habilitar y/o bloquear puertos individualmente y por conjuntos.

Ejemplo: Todos los puertos inferiores al 1024 están bloqueados, todos los superiores están habilitados salvo el 8080. Además, los puertos 80, 23, 25 y 110 deben estar habilitados.

Filtrado de IPs: debe permitir filtrar la IP o hostname de destino y de origen, tanto individualmente como por rangos.

Ejemplos:

- No se aceptará ningún mensaje proveniente de la IP 24.35.126.155.
- La IP 192.168.0.19 no podrá recibir ningún mensaje proveniente de la 24.56.178.165.
- Las IP desde la 192.168.0.1 hasta la 192.168.0.10 no pueden recibir mensajes de la 24.35.126.154.

Acción ante el bloqueo: Si se bloquea un mensaje de entrada, se puede tomar la decisión de enviar un mensaje a la IP origen informando que el envío no fue exitoso. También se puede tomar la decisión de loguear (Inicialmente pensemos en informar por consola) que el mensaje fue bloqueado.

Ejemplos:

- No se aceptará ningún mensaje proveniente de la IP 24.35.126.154 y se loguearán todos los mensajes recibidos desde esa IP.
- La IP 192.168.0.19 no podrá recibir ningún mensaje y se debe informar al origen que se denegó la entrega del mensaje.

Otras acciones:

Se puede decidir loguear un mensaje a enviar, aún cuando se permita enviarlo.

Ejemplo:

- Todos los mensajes al puerto 1521 deben loguearse.

Se puede decidir realizar un forward de una IP/puerto

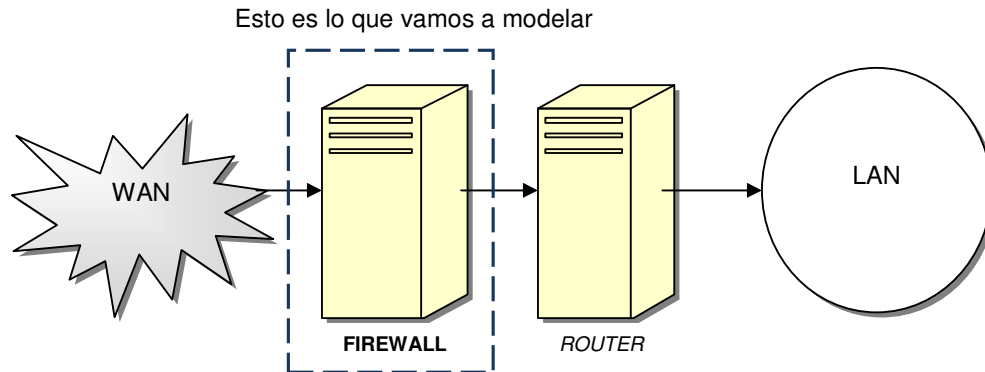
Ejemplo:

- Los mensajes a la IP 192.168.0.1 puerto 80, deben ser redirigidos a la IP 192.168.0.4 puerto 8866.

Si bien no se considerarán en esta iteración, en futuras iteraciones aparecerán nuevas posibles acciones.

Explicación del dominio

¿Qué es un Firewall?



El Firewall puede ser una combinación de hardware + software que filtra el pasaje de paquetes desde una red WAN (Internet o cualquier red externa) hacia una LAN. El Firewall habla con el Router que se encarga de hacer llegar los paquetes de red a cada puesto de trabajo de una LAN.

Pensamos cómo sería la aplicación que maneja el Firewall: primer punto, nos encontramos con un dominio abstracto.

- No son vacas, rinocerontes ni personas
- Tampoco facturas o ítems de factura (que si bien no son cosas tangibles, son conceptos más concretos)

La idea es que el Firewall se pueda configurar y que pueda jugar con las reglas

Primeras ideas

Damos 15 minutos para pensar cómo encararlo (así todos pueden intentar tirar ideas y compararlas con las de los demás). Preguntamos entonces de qué manera encararon el ejercicio. Aparecieron varias alternativas:

- 1) Pensar en los objetos candidatos, en particular, partir del objeto Firewall y buscar las relaciones con los otros objetos posibles:
 - Paquete (IP origen, IP destino)
 - Dirección
 - Router
 - Puertos
 - Bloqueo, que después renombramos a Regla
 - Tipo de regla (abstrayendo validaciones posibles: filtrar IP destino, filtrar IP origen, etc.)
- 2) Otra opción era pensarlo a partir de los requerimientos del usuario, que pueden ser dos:
 - Crear una regla
 - Responder a la llegada de un paquete

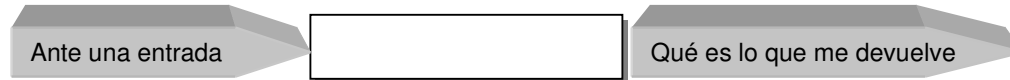
Después vimos que el enunciado no pedía explícitamente crear una regla, entonces en este caso particular no me agrega mucho pensar en los requerimientos porque hay un solo caso de uso...

- 3) Aplicar Test Driven Development (TDD) y pensar en qué Tests hay que definir.

Requerimiento es cualquier cosa que el usuario me puede pedir:

- Que los paquetes que se envían a la IP 192.32.18.126 se rechacen
 - Loguear todos los paquetes que envía la IP 10.122.192.128
- etc.

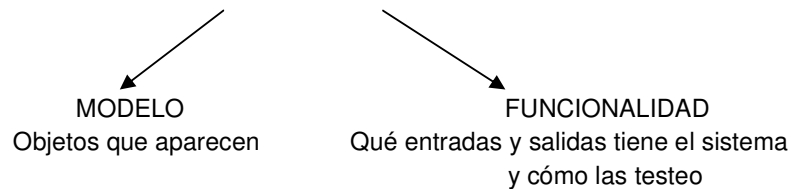
Estamos pensando en ejemplos, lo que los libros de UML llaman *escenarios de un caso de uso*. Lo que me interesa es:



2) y 3) me llevan a:

- Identificar qué escenarios puede haber y
- Cómo los testeo

Entonces volvemos sobre el Proceso de Diseño y decimos que hay **dos visiones** complementarias que tenemos que tener para resolver un problema:



Empiezo entonces por la visión de modelo y vamos a volver a la lista de objetos posibles. Encontramos algunos más:

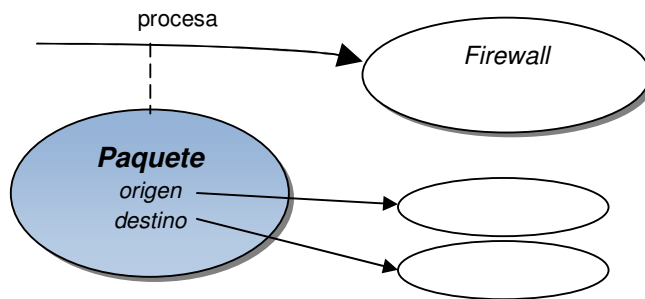
- Firewall
- Paquete
- Dirección IP
- Router
- Puerto
- Regla
- Tipo de Regla
- Configuración
- Acción/Decisión
- Redirección
- Log
- Wan
- Persona

Nos tomamos una pequeña licencia para abrir la cancha: ¿qué alternativas tengo en objetos para construir los conceptos Firewall, Regla, Tipo de Regla, etc.?

- Clase
- Interfaz
- Método
- Relación/Atributo
- Mensaje
- Subclase
- Instancia u objeto

Value Objects, God Objects

Armamos una especie de diagrama de colaboración. Pensamos en el Firewall y en las relaciones que ese objeto tiene con otros objetos posibles:



¿Qué responsabilidades tiene el Paquete?

- Conocer su IP origen
- Conocer su IP destino
- Conocer su contenido

¿No es al fin y al cabo una estructura de datos? Sí, Paquete es un **Value Object**, su finalidad es encapsular (agrupar) valores. Esto es mejor para el Firewall que recibir todos los datos sueltos, así que no está mal que algunos de los objetos de mi dominio no tengan mayor comportamiento que éste. Lo *peligroso* (y para nada deseable) es que *todos* los objetos de mi sistema sean *ValueObjects* y que haya un solo objeto (el Firewall en este caso) que tenga todo el comportamiento.

Si así fuera, el Firewall actuaría como un súper main() gigante, y el Firewall sería un **God Object**: un objeto que es responsable de todo el sistema. El problema principal de los God Objects es que van en contra de las ideas del paradigma de objetos en cuanto a distribuir responsabilidades y dejar que cada objeto haga lo que tiene que hacer (y nada más).

El God Object es un **Antipattern**: un ejemplo de algo que no hay que hacer (ideas que no conviene aplicar).

En el paquete, ¿origen y destino son objetos?

Sí, de qué tipos

- String
- Dirección: para qué me sirve que sea un objeto → si necesito que tenga comportamiento.

A esta altura no hay argumentos suficientes para decidirme por alguna opción. Entonces lo dejo pendiente...

Modelando las reglas

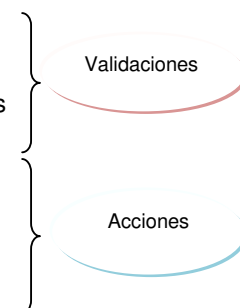
La regla puede ser una clase abstracta o interface. Tenemos muchas validaciones posibles.

¿Cómo elegimos el criterio para subclasificar los tipos de regla?

- Agrupamos las que trabajan por IP / por rango de IPs
- Agrupamos las que trabajan por puerto / por rango de puertos
- Agrupamos las que trabajan por IP + puerto / rango de IPs + puertos

¿Qué consecuencias puede traer cada una de las validaciones?

- a) Redirección a otra IP
- b) Bloqueo (descarto el paquete) o habilitación
- c) Log
- d) Informe del envío no exitoso



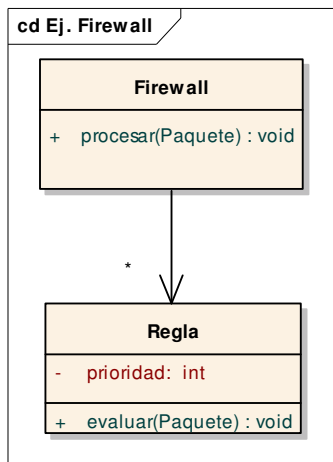
Validaciones + Acciones → ¿Cómo las traduzco a objetos?

El Firewall tiene reglas y cada regla tiene asociadas acciones...

Ok, llegamos hasta aquí: ahora vamos a mostrar una solución posible. Esta solución no es ideal, porque **no hay una solución ideal para este tipo de ejercicios (esta es una diferencia sustancial con Paradigmas)**. Todas las soluciones son criticables y tienen alguna debilidad. El punto es que yo en el examen pueda entender qué debilidades tiene mi solución, pero bancármela y elegir ésta porque tiene mayores beneficios que otras posibilidades. Basta con que pueda defender mi postura y que mi solución sea coherente.

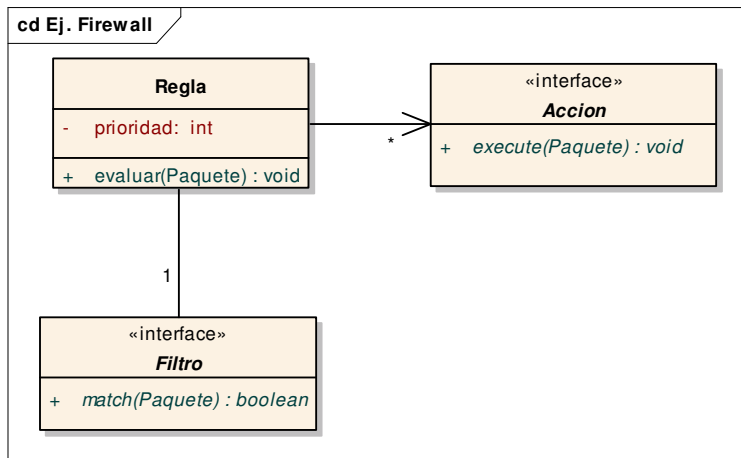
Una solución

El firewall tiene un conjunto de reglas.



Cada regla entiende el mensaje evaluar(Paquete paquete), que delega a dos objetos:

- Un **filtro** que sabe si el paquete encaja con esta regla. El mensaje que le envía es `match(Paquete)` y devuelve un booleano.
- Una **acción** (o varias) a ejecutar en caso de que haya matching. El mensaje que le envía es `execute(Paquete)`

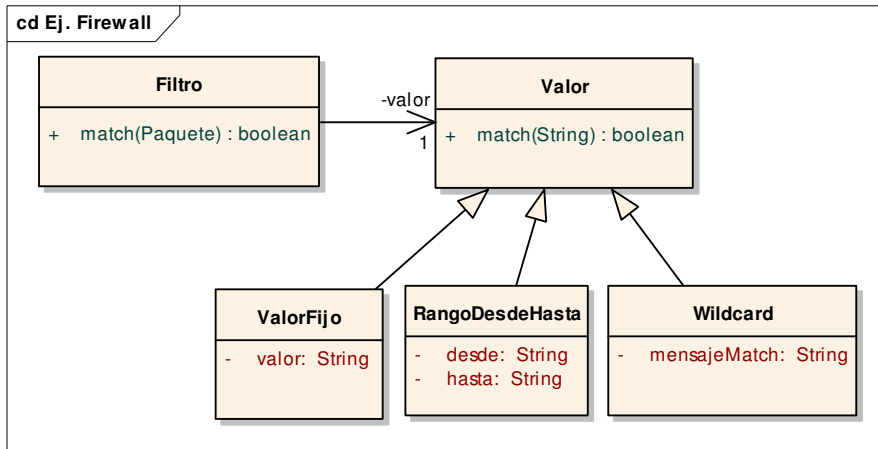


Ejemplo: si la regla define que se debe logear todos los mensajes que se envían a la IP 198.20.172.128

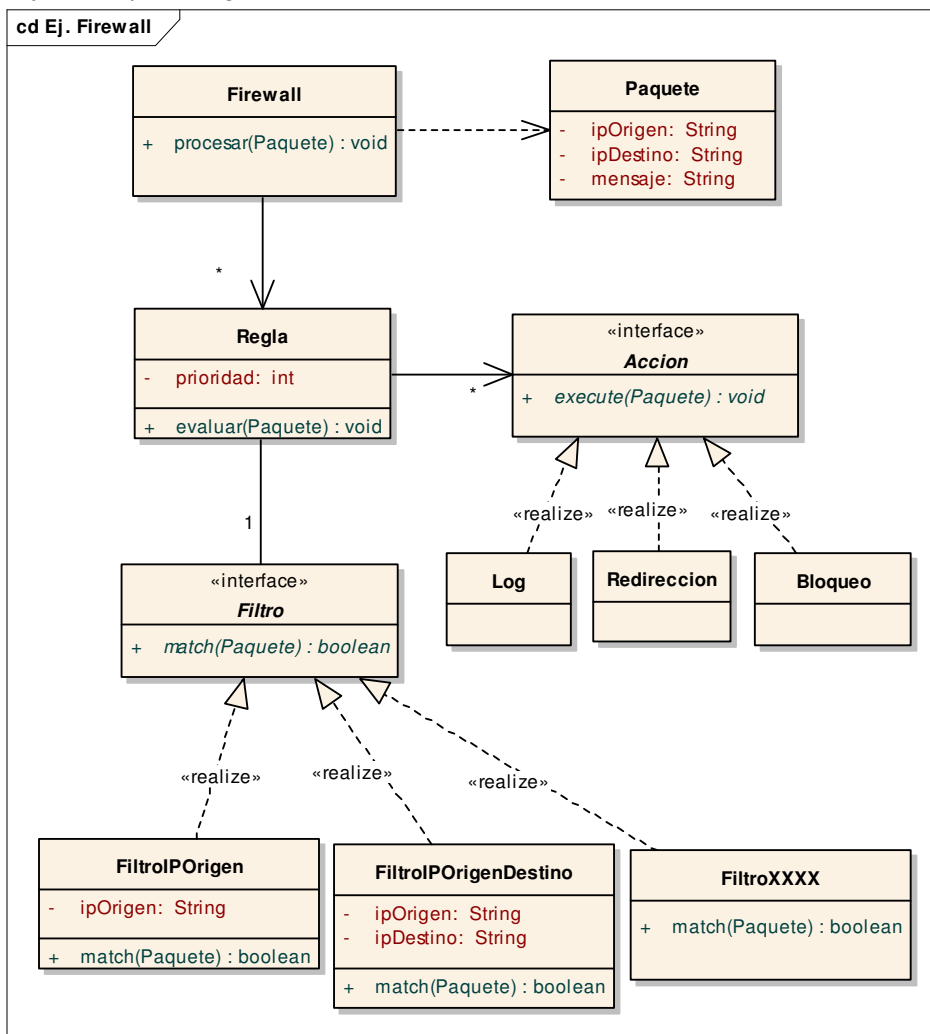
- El filtro por IP debe tener en IP Destino el valor "198.20.172.128".
- La acción asociada (el `execute()`) debe ser hacer el Sysout correspondiente.

La decisión de separar a la regla y al filtro es sobrediseño: ok, estoy dispuesto a aceptar ese costo por el beneficio que me puede dar separar ambos conceptos.

Los filtros se subclasifican por tipo de filtro: FiltroIPOrigen, FiltroIPDestino, FiltroIPOrigenDestino, FiltroPuertoOrigen, etc. El rango se puede modelar con un objeto al que le paso una IP y me dice si matchea o no, también lo puedo modelar con una IP que admita wildcards: 168.192.1.*



Dejamos aquí el diagrama de clases:



Las reglas están ordenadas por prioridad (y es muy probable que se pisen): la primera regla que aplica es la que determina qué acciones se ejecutan.

Tenemos que ver cómo manejamos el envío del mensaje:

- 1) Lo hacemos dentro del código mismo
- 2) Metemos al envío dentro de la colección de Reglas con una acción más que sea Envío.

Cualquiera de las dos decisiones que tomemos nos lleva a que tenemos que manejar el bloqueo y el forward de los mensajes porque en ese caso no tenemos que enviar el paquete.

Aquí nuevamente hay opciones para manejar el no-envío del paquete:

- 1) Modificamos la firma del método evaluar() de la Regla para que devuelva un booleano y así saber si hay que seguir evaluando las reglas.
- 2) Alteramos el flujo normal de evaluación de reglas con una excepción.

Si elegimos 2-1 (el envío es una regla más que tiene la última prioridad y el método evaluar() devuelve un booleano que indica si se deben seguir procesando las siguientes reglas), el método procesar() queda:

```
public void procesar(Paquete paquete) {  
    for (Regla regla : this.reglasOrdenadasPorPrioridad()) {  
        if (!regla.evaluar(paquete)) {  
            break;  
        }  
    }  
}
```

Si desde la acción de bloqueo y forward tiramos una excepción, el método procesar(Paquete) en Firewall tiene la siguiente lógica:

```
public void procesar(Paquete paquete) {  
    try {  
        for (Regla regla : this.reglasOrdenadasPorPrioridad()) {  
            regla.evaluar(paquete); 2  
        }  
        1  
    } catch (FinalizarReglaException e) {  
        // ...  
    }  
}
```

El código que hay que agregar para trabajar con el envío está marcado en 1) –lo que sigue al for each- o está incorporado en 2) como una Regla más.

Nos posicionamos en la clase Regla. Tenemos un filtro asociado y un conjunto de acciones. ¿Me importa el orden en el que lo guardo? Sí, porque no es lo mismo que loguee y después bloquee que viceversa, porque si bloqueo primero puede no ejecutarse la siguiente acción. Entonces voy a modelar con una *lista* las acciones. Codificamos el método evaluar:

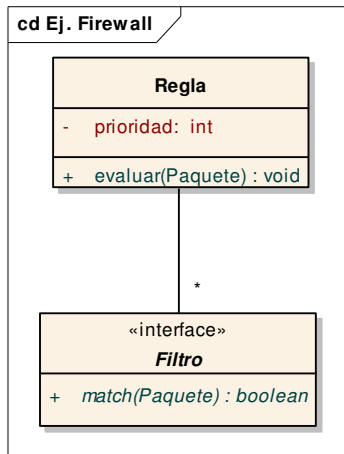
```
public void evaluar(Paquete paquete) {  
    if (filtro.match(paquete)) {  
        for (Accion accion : acciones) {  
            accion.execute(paquete);  
        }  
    }  
}
```

Si tiro una excepción en la acción:

- Por un lado es mucho más cómodo trabajar dentro de evaluar porque no es necesario capturar el valor booleano que devuelva `accion.execute(paquete)` y devolverlo a su vez en el mismo método `evaluar()`
- Por otro lado tengo que tener en cuenta que no tiene sentido configurar una regla que tenga dos acciones: bloquear y loguear como segunda acción porque nunca se va a ejecutar.

Combinando reglas

Podríamos modelar a la regla para que admita más de un filtro/condición/criterio:



Esto nos permitiría resolver este requerimiento (Caso 1: AND):

- Hay que loguear y bloquear todos los mensajes donde
 - la IP de destino sea 120.136.1.18
 - **y donde** la IP de origen 148.1.29.28

De todos modos, podría ser que otra regla tuviera este formato (Caso 2: OR)

- Todos los mensajes cuya IP de origen comienza con 120.136
- o aquellos cuya IP de destino es 148.1.49.*
 - deben ser redirigidas a la IP 160.182.192.12

¿Cómo atacamos la posibilidad de definir reglas con Y o con O? Es decir, si tengo una colección solamente no me alcanza, así tengo que definir que **todos** los filtros se cumplan (AND), o bien que **alguno** de los filtros se cumplan (OR).

Una opción es modelar el OR como dos reglas separadas. En el caso 2 podríamos resolverlo con dos objetos Regla:

- Un objeto regla con un filtro que matchee los mensajes cuya IP de Origen comienza con 120.136
- Otro objeto regla con un filtro que matchee los mensajes cuya IP destino es 148.1.49.*

La contra de esta solución es que debo repetir la misma acción (forwardear a la misma IP).

¿Entonces? ¿Cómo puedo modelar las condiciones configurando yo el AND/OR? Revisemos el caso 2 nuevamente:

- [FILTRO Todos los mensajes cuya IP de origen comienza con 120.136]
- o [FILTRO aquellos cuya IP de destino es 148.1.49.*]
 - [ACCION deben ser redirigidas a la IP 160.182.192.12]

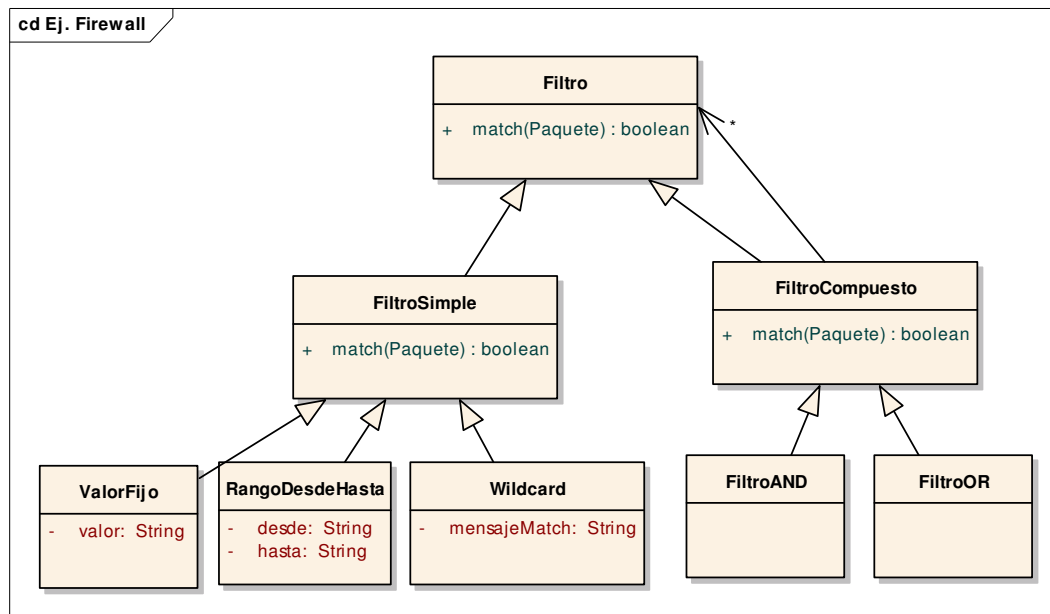
Y el caso 1:

- [ACCION Hay que loguear] y [ACCION bloquear] todos los mensajes donde
 - [FILTRO la IP de destino sea 120.136.1.18]
 - **y donde** [FILTRO la IP de origen 148.1.29.28]

Lo que no quedó como objeto fue el o, y el y. ¿Qué tal si pudiéramos modelar la condición como un objeto más? Ok, un objeto OR ¿qué necesita? Dos operadores, que son filtros. Lo mismo pasa con el AND. O sea, el OR y el AND son criterios compuestos:

- Un filtro matchea si cumple la condición
- Un OR matchea si se cumple alguna de las condiciones que conoce. Cada condición puede ser:
 - Un filtro
 - Otra condición OR/AND (como si fueran paréntesis: (A AND (B OR C)) OR D
- Un AND matchea si se cumplen todas las condiciones que conoce. Cada condición puede ser:
 - Un filtro
 - Otra condición OR/AND (como si fueran paréntesis: (A AND (B OR C)) OR D

O sea tenemos filtros simples y compuestos. Podemos aplicar un *Composite* para trabajar el método match() en forma polimórfica:



Configurando la regla para el envío de un mensaje

Desarrollemos un poco la idea de trabajar con una regla default para el envío.

Ese objeto tiene que estar configurado con un filtro y con una acción asociada:

- El filtro devuelve el valor fijo true en el método match (no me importa ip origen ni destino):

#FiltroDefault

```
public boolean match(Paquete paquete) {
    return true;
}
```

- tiene como acción asociada enviar el paquete

#Envío (hereda de Accion)

```
public void execute(Paquete paquete) {  
  
}
```

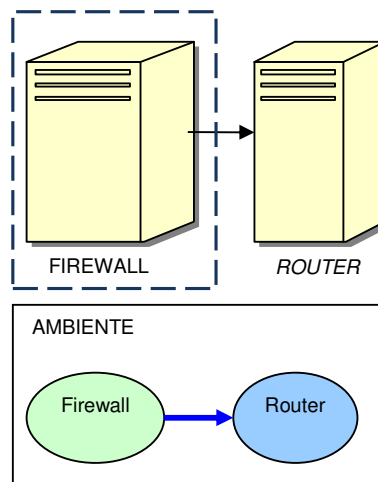
Modelando el envío de un paquete (intro breve a interfaces)

¿Cómo modelamos el envío de un paquete?

¿Qué objeto es responsable de esto?

Revisemos el enunciado. Vemos que en la realidad el router es el que direcciona los paquetes a cada IP.

¿Me sirve trabajar con un Router como abstracción? Sí, me sirve porque desde el Firewall yo no tengo que conocer la lógica de cómo se implementa el direccionamiento de los paquetes. De hecho el Router real puede proveerme una función para que por software le envíe paquetes. Lo que sí tengo que hacer es modelar la interfaz entre el Firewall y el Router y me gustaría que fuera con objetos.



¿Cómo le pido algo a alguien externo? Si el Router está modelado como un objeto, le puedo mandar un mensaje y listo, tan simple como eso es “modelar la interfaz con el Router”. Definimos:

- el mensaje que vamos a enviarle
- qué parámetros necesita

¿Qué tal send(Paquete)?

Buenísimo, ahora cómo conocemos al Router desde la acción Envío:

- Es un atributo del Envío: mmm... puede ser pero hay que configurarlo dentro de las reglas
- Me lo pasan como parámetro: agh, ¿cambiar la firma del método execute(), agregando un parámetro más? Busquemos otra opción
- Se lo pido al Firewall ¿Cómo? Podría ser un getRouter()
- Accedo globalmente a través de un método estático de Router (algo similar a lo anterior)

En clases posteriores vamos a ver cómo implementar esto, pensemos que hay un método feliz que me devuelve el único objeto Router que hay en el ambiente y se lo pido a la clase Router: Router.getRouter(). Entonces el método execute() de envío queda así:

```
public void execute(Paquete paquete) {  
    Router.getRouter().send(paquete);  
}
```

Lo bueno y lo malo de configurar varias acciones

Las acciones las manejamos como una lista porque eso me permite configurar:

- Una regla que tenga como acciones loguear y enviar el mensaje
- O loguear y bloquear el mensaje
- O bloquearlo

etc.

La contra de esta solución (y por eso no es ideal) es que tengo que evitar que cuando la instancie me genere acciones inconsistentes para el negocio: por ejemplo que no habilite y bloquee el mensaje a la vez:

```
Regla reglaLoca = new Regla();
```

```
reglaLoca.addAction(new Bloqueo());
```

```
reglaLoca.addAction(new Habilitacion()); No tiene sentido
```

Para eso vamos a trabajar en breve los patrones que me pueden servir para instanciar objetos, mientras les tiramos dos ideas posibles para que esto no ocurra:

- 1) reglaLoca.addAction(new BloqueoYLogueo());
- 2) FactoryAcciones.crearAccionBloqueoYLogueo();

Interfaces del sistema

Si pensamos en la aplicación que acabamos de diseñar como un componente de un sistema más grande, podemos pensar en

- qué me piden (qué servicios brindo) → Interfaces de entrada y
- qué necesito (qué servicios uso) → Interfaces de salida

Las interfaces las vamos a modelar como mensajes a objetos, más allá de que no necesariamente haya del otro lado un componente desarrollado con lenguajes OO.

Interfaz de entrada con el firewall

+ procesar(Paquete paquete)

Interfaz de salida que usa el firewall

+ enviar un mensaje (ya lo hicimos, hablando con el Router)

+ notificar el envío/rechazo de un mensaje

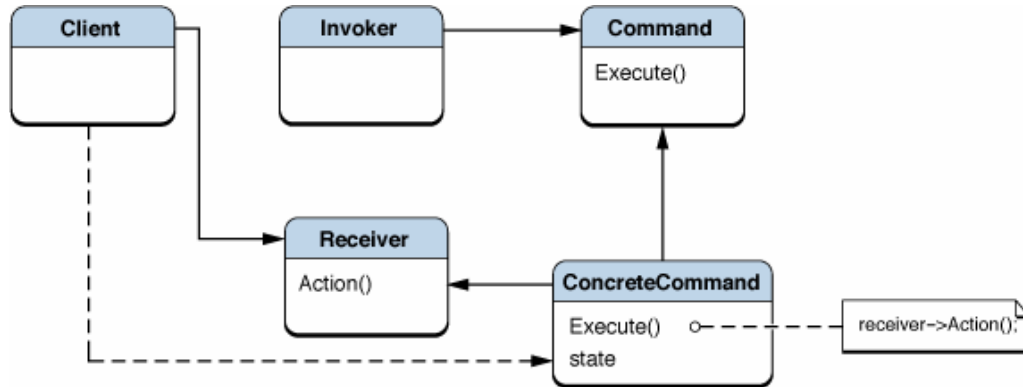
¿Cómo modelamos esto? Podemos agregar una acción que informe a la dirección IP origen que el envío no fue exitoso. Para ello vamos a tener que configurar una regla que tenga:

- 1) Como primera acción el “informe de envío no exitoso”
- 2) La acción de bloqueo como segunda acción (recordemos que no puede estar primero porque de lo contrario no informa el envío no exitoso a la IP origen, si me empieza a molestar mucho puedo decidir cambiar la lógica del método evaluar()).

Desarrollamos el método para la acción InformarEnvíoNoExitoso:

```
public void execute(Paquete paquete) {  
    Paquete informe = new Paquete  
        (Firewall.getIPOrigen(), ← IP origen  
         paquete.getIpOrigen(), ← IP destino  
         "Envío no exitoso"); ← mensaje  
    Router.getRouter().send(informe);  
}
```

El Command Pattern



La manera en que modelamos las acciones terminan describiendo el patrón de diseño Command, que nos permite abstraer una porción de código a ejecutar. Se parece al Strategy, es verdad, pero la intención que tiene es diferente:

- Los strategies son algoritmos que puedo intercambiar, los commands son acciones que puedo diferir –pasándolos como parámetros a otros objetos- o ejecutar cuando yo quiera.
- Los commands facilitan guardar en un historial las operaciones efectuadas (y eventualmente reprocesar acciones a partir de un punto).
- Los commands se pueden combinar y agrupar (bloqueo, log y bloqueo, log y redireccionamiento, incluso alterando el orden, etc.) ¿Qué sucede respecto al cliente y los strategies en los ejemplos que hemos visto en la cursada?
- Los commands podrían implementar un método undo() para deshacer los efectos del execute(). No es éste el caso, pero es una diferencia respecto al Strategy.